FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Julius Jojo C. Sabulao Jr
Section:	An43
Professor:	Crisola Tan
Date:	10/11/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

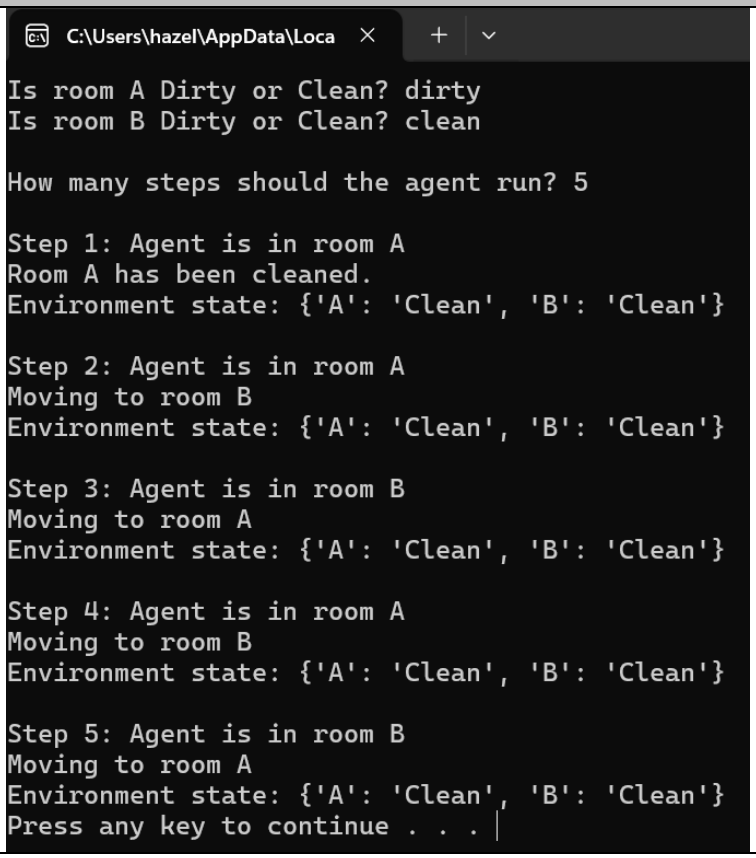
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

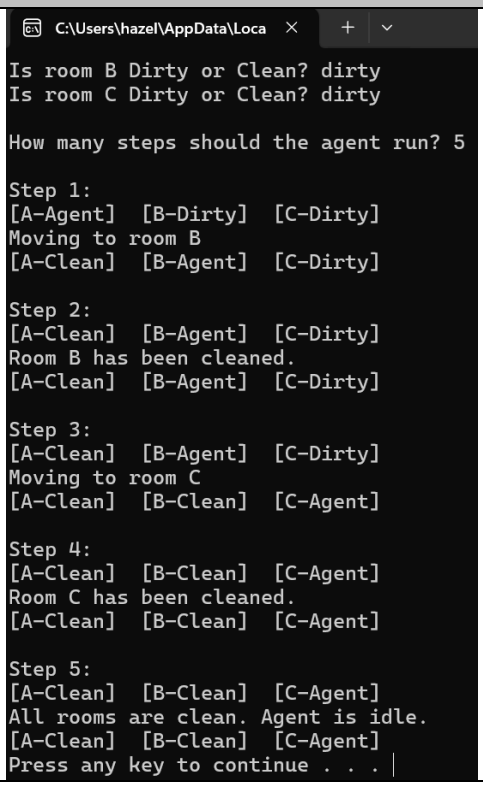
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
from dataclasses import dataclass
from pathlib import Path


@dataclass
class TwoRoomEnvironment:
    """Stores and updates the condition of rooms A and B."""

    rooms: dict[str, str]
```

```
    def perceive(self, location: str) -> str:
        return self.rooms[location]

    def is_dirty(self, location: str) -> bool:
        """Return True when the selected room is dirty."""
        return self.rooms[location] == "Dirty"

    def clean(self, location: str) -> None:
        self.rooms[location] = "Clean"

class RuleBasedVacuumAgent:
    """Chooses actions using predefined condition-action rules."""

    def __init__(self, starting_location: str):
        self.location = starting_location

    def perceive_and_act(self, environment: TwoRoomEnvironment) -> str:
        """Apply the required rules and return a readable action."""
        # Rule 1: If the current room is dirty, clean it.
        if environment.is_dirty(self.location):
            environment.clean(self.location)
            return f"CLEANED Room {self.location}"

        # Rule 2: If the current room is clean, move to the other room.
        previous_location = self.location
        self.move()
        return f"MOVED from Room {previous_location} to Room {self.location}"

    def move(self) -> None:
        """Switch between Room A and Room B."""
        self.location = "B" if self.location == "A" else "A"


def normalize_status(value: str) -> str:
    value = value.strip().lower()
    if value == "dirty":
        return "Dirty"
    if value == "clean":
        return "Clean"
    raise ValueError("Room status must be Dirty or Clean.")


def run_simulation(room_a: str, room_b: str, start: str, number_of_steps: int):
```

```
    environment = TwoRoomEnvironment({"A": room_a, "B": room_b})
    agent = RuleBasedVacuumAgent(start)
    records = []

    print("\nRULE-BASED VACUUM AGENT")
    print("Step | Location Before | Perception | Action                    | Room A | Room B | Location
After")
    print("-" * 108)

    for step in range(1, number_of_steps + 1):
        location_before = agent.location
        perception = environment.perceive(location_before)
        action = agent.perceive_and_act(environment)

        record = {
            "step": step,
            "location": location_before,
            "perception": perception,
            "action": action,
            "room_a": environment.rooms["A"],
            "room_b": environment.rooms["B"],
            "location_after": agent.location,
        }
        records.append(record)

        print(
            f"{step:>4} | {location_before:^15} | {perception:^10} | "
            f"{action:<29} | {environment.rooms['A']:^6} | "
            f"{environment.rooms['B']:^6} | {agent.location:^14}"
        )

    return environment, agent, records


def save_results(records, filename="TA2_simulation_results.txt"):
    lines = [
        "TA2 - RULE-BASED VACUUM AGENT RESULTS",
        "",
        "Step | Location Before | Perception | Action | Room A | Room B | Location After",
        "-" * 108,
    ]

    for row in records:
        lines.append(
```

```
            f"{row['step']:>4} | {row['location']:^15} | {row['perception']:^10} | "
            f"{row['action']:<29} | {row['room_a']:^6} | {row['room_b']:^6} | "
            f"{row['location_after']:^14}"
        )

    Path(filename).write_text("\n".join(lines), encoding="utf-8")


def main():
    print("TA2 - RULE-BASED AGENT")
    print("Enter either Dirty or Clean for each room.")

    try:
        room_a = normalize_status(input("Initial condition of Room A: "))
        room_b = normalize_status(input("Initial condition of Room B: "))
        start = input("Starting location (A/B): ").strip().upper()
        number_of_steps = int(input("Number of simulation steps: "))

        if start not in {"A", "B"}:
            raise ValueError("Starting location must be A or B.")
        if number_of_steps <= 0:
            raise ValueError("Number of steps must be greater than zero.")

        environment, agent, records = run_simulation(
            room_a, room_b, start, number_of_steps
        )
        save_results(records)

        print("\nFINAL RESULT")
        print(f"Room A: {environment.rooms['A']}")
        print(f"Room B: {environment.rooms['B']}")
        print(f"Agent location: {agent.location}")
        print("Results saved to TA2_simulation_results.txt")

    except ValueError as error:
        print(f"Input error: {error}")


if __name__ == "__main__":
    main()
```

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

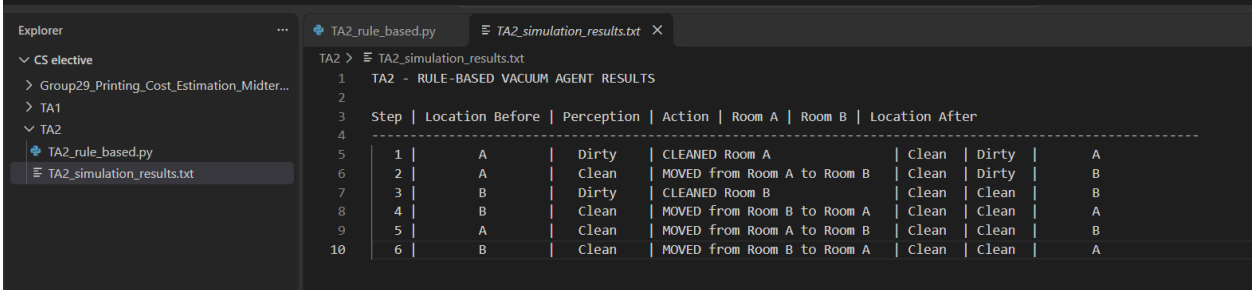
Answer the Questions ⇩

1. The agent follows two predefined condition-action rules. First, if the agent perceives that its current room is dirty, it cleans the room. Second, if the current room is already clean, it moves to the other room. The agent applies these rules during every simulation step.
2. The agent checks or perceives the condition of its current room. If the room is marked Dirty, the agent performs the cleaning action and changes its condition to Clean. If the room is already Clean, the agent moves to the other room. Its decision depends entirely on the current room's condition.
3. If both rooms are already clean, the agent does not perform any cleaning action. Based on the required rules, it continues moving between Room A and Room B for the specified number of simulation steps because every room it perceives is already clean. The environment remains unchanged.
4. To handle more than two rooms, I would store all rooms in a dictionary or list, such as Rooms A, B, and C. The agent's movement method could cycle through the rooms or search for the next dirty room. After perceiving a clean room, the agent would move to another room; after perceiving a dirty room, it would clean it. The program could also allow the agent to randomly select a dirty room or stop after all rooms have been cleaned.

```
PS C:\Users\Msi\Documents\School\CS elective\TA2> python .\TA2_rule_based.py
TA2 - RULE-BASED AGENT
Enter either Dirty or Clean for each room.
Initial condition of Room A: Dirty
Initial condition of Room B: Dirty
Starting location (A/B): a
Number of simulation steps: 6

RULE-BASED VACUUM AGENT
Step | Location Before | Perception | Action                        | Room A | Room B | Location After
-------------------------------------------------------------------------------------------------------------
   1 |        A        |   Dirty    | CLEANED Room A                | Clean  | Dirty  |       A
   2 |        A        |   Clean    | MOVED from Room A to Room B   | Clean  | Dirty  |       B
   3 |        B        |   Dirty    | CLEANED Room B                | Clean  | Clean  |       B
   4 |        B        |   Clean    | MOVED from Room B to Room A   | Clean  | Clean  |       A
   5 |        A        |   Clean    | MOVED from Room A to Room B   | Clean  | Clean  |       B
   6 |        B        |   Clean    | MOVED from Room B to Room A   | Clean  | Clean  |       A

FINAL RESULT
Room A: Clean
Room B: Clean
Agent location: A
Results saved to TA2_simulation_results.txt
PS C:\Users\Msi\Documents\School\CS elective\TA2>
```



The agent successfully perceived and cleaned both dirty rooms. During the first step, it detected that Room A was dirty and cleaned it. Since Room A was clean during the second step, it moved to Room B. It then detected that Room B was dirty and cleaned it. After both rooms became clean, the agent continued moving between Rooms A and B until the specified number of simulation steps was completed. This demonstrates that the agent consistently follows predefined condition-action rules.